

---

# Guide Essentiel d'Algorithmique Avancée

Complexité, diviser pour régner, programmation dynamique,  
algorithmes gloutons, graphes et structures de données avancées

Rapport pédagogique — 8 pages

---

# 1. Analyse de complexité

L'algorithmique avancée repose sur la capacité à évaluer l'efficacité d'un algorithme, indépendamment du matériel, en analysant comment son temps d'exécution et sa consommation mémoire évoluent avec la taille de l'entrée  $n$ .

## Notation Big O

La notation  $O(\dots)$  décrit la borne supérieure de croissance d'un algorithme dans le pire des cas.

|                                 |                                                           |
|---------------------------------|-----------------------------------------------------------|
| <b><math>O(1)</math></b>        | Temps constant — accès à un tableau par index.            |
| <b><math>O(\log n)</math></b>   | Logarithmique — recherche dichotomique.                   |
| <b><math>O(n)</math></b>        | Linéaire — parcours simple d'une liste.                   |
| <b><math>O(n \log n)</math></b> | Quasi-linéaire — tri fusion, tri rapide (moyenne).        |
| <b><math>O(n^2)</math></b>      | Quadratique — boucles imbriquées, tri à bulles.           |
| <b><math>O(2^n)</math></b>      | Exponentielle — recherche exhaustive (backtracking naïf). |

## Théorème général (Master Theorem)

Pour une relation de récurrence  $T(n) = a \cdot T(n/b) + f(n)$ , le théorème général permet de déduire directement la complexité d'un algorithme récursif de type diviser-pour-régner, en comparant  $f(n)$  à  $n$  élevé à la puissance  $\log_b a$ .

$$T(n) = a \cdot T(n/b) + f(n)$$

$$\text{Cas 1 : } f(n) \text{ domine } n^{\log_b a} \Rightarrow T(n) = \Theta(n^{\log_b a})$$

$$\text{Cas 2 : } f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

$$\text{Cas 3 : } n^{\log_b a} \text{ domine } f(n) \Rightarrow T(n) = \Theta(f(n))$$

Exemple : le tri fusion vérifie  $T(n) = 2T(n/2) + O(n)$ , soit le cas 2  $\rightarrow$  complexité  $O(n \log n)$ .

## 2. Diviser pour régner (Divide & Conquer)

Cette technique consiste à décomposer un problème en sous-problèmes plus petits de même nature, à les résoudre récursivement, puis à combiner leurs résultats.

### Tri fusion (Merge Sort) — $O(n \log n)$

```
def merge_sort(tab):
    if len(tab) <= 1:
        return tab

    milieu = len(tab) // 2
    gauche = merge_sort(tab[:milieu])
    droite = merge_sort(tab[milieu:])
    return fusionner(gauche, droite)

def fusionner(g, d):
    resultat = []
    i = j = 0
    while i < len(g) and j < len(d):
        if g[i] <= d[j]:
            resultat.append(g[i]); i += 1
        else:
            resultat.append(d[j]); j += 1
    return resultat + g[i:] + d[j:]
```

### Recherche dichotomique — $O(\log n)$

```
def recherche_binaire(tab, cible):
    gauche, droite = 0, len(tab) - 1
    while gauche <= droite:
        milieu = (gauche + droite) // 2
        if tab[milieu] == cible:
            return milieu
        elif tab[milieu] < cible:
            gauche = milieu + 1
        else:
            droite = milieu - 1
    return -1 # non trouvé
```

Le tri rapide (Quick Sort), le calcul rapide de puissance et la multiplication de Karatsuba suivent également ce paradigme.

### 3. Programmation dynamique (DP)

La programmation dynamique résout un problème en le décomposant en sous-problèmes **chevauchants**, en mémorisant leurs résultats (mémoïsation) pour éviter les recalculs. Elle s'applique lorsque le problème présente une sous-structure optimale.

#### Exemple : suite de Fibonacci mémoïsée

```
memo = {}

def fib(n):
    if n <= 1:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]

# Complexité : O(n) au lieu de O(2^n) en version naïve récursive
```

#### Exemple : sac à dos (Knapsack) 0/1

```
def sac_a_dos(poids, valeurs, capacite):
    n = len(poids)
    dp = [[0] * (capacite + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for c in range(capacite + 1):
            if poids[i-1] <= c:
                dp[i][c] = max(dp[i-1][c],
                               dp[i-1][c-poids[i-1]] + valeurs[i-1])
            else:
                dp[i][c] = dp[i-1][c]

    return dp[n][capacite] # complexité O(n * capacite)
```

- Autres applications classiques : plus longue sous-séquence commune (LCS), distance d'édition, rendu de monnaie, chemin minimal dans une grille.

## 4. Algorithmes gloutons (Greedy)

Un algorithme glouton construit une solution étape par étape, en choisissant à chaque fois l'option localement optimale, sans revenir en arrière. Il est rapide, mais ne garantit une solution globalement optimale que pour certains problèmes précis.

### Quand utiliser une approche gloutonne ?

- Le problème possède la propriété de **choix glouton** : un choix localement optimal mène à une solution globalement optimale.
- Le problème a une **sous-structure optimale** similaire à la DP.
- Exemples typiques : rendu de monnaie (avec certains systèmes), arbre couvrant minimal (Kruskal, Prim), codage de Huffman, ordonnancement d'intervalles.

### Exemple : sélection d'intervalles compatibles

Objectif : sélectionner un nombre maximal d'intervalles qui ne se chevauchent pas.

```
def selection_intervalles(intervalles):  
    # Trier par date de fin croissante (choix glouton clé)  
    intervalles.sort(key=lambda x: x[1])  
  
    resultat = []  
    fin_precedente = float('-inf')  
  
    for debut, fin in intervalles:  
        if debut >= fin_precedente:  
            resultat.append((debut, fin))  
            fin_precedente = fin  
  
    return resultat # complexité  $O(n \log n)$ , dominée par le tri
```

*Attention : contrairement à la DP, le glouton ne réexamine jamais un choix — il faut donc prouver formellement son optimalité avant de l'utiliser.*

## 5. Algorithmes de graphes

### Parcours en largeur (BFS) et en profondeur (DFS)

Le BFS explore un graphe niveau par niveau (via une file), utile pour trouver le plus court chemin en nombre d'arêtes. Le DFS explore en profondeur (via une pile ou la récursion), utile pour détecter des cycles ou faire un tri topologique.

```
from collections import deque

def bfs(graphe, depart):
    visites = {depart}
    file = deque([depart])
    ordre = []
    while file:
        sommet = file.popleft()
        ordre.append(sommet)
        for voisin in graphe[sommet]:
            if voisin not in visites:
                visites.add(voisin)
                file.append(voisin)
    return ordre # complexité  $O(V + E)$ 
```

### Plus court chemin : algorithme de Dijkstra

Dijkstra calcule le plus court chemin depuis une source, dans un graphe pondéré à poids positifs, grâce à une file de priorité (tas min).

```
import heapq

def dijkstra(graphe, depart):
    distances = {s: float('inf') for s in graphe}
    distances[depart] = 0
    file = [(0, depart)]

    while file:
        d, sommet = heapq.heappop(file)
        if d > distances[sommet]:
            continue
        for voisin, poids in graphe[sommet]:
            nd = d + poids
            if nd < distances[voisin]:
                distances[voisin] = nd
                heapq.heappush(file, (nd, voisin))
    return distances # complexité  $O((V + E) \log V)$ 
```

## 6. Structures de données avancées

|                                         |                                                                                                          |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Tas (Heap)</b>                       | Arbre binaire permettant d'extraire le min/max en $O(\log n)$ — base des files de priorité.              |
| <b>Arbre binaire de recherche (BST)</b> | Recherche, insertion, suppression en $O(\log n)$ en moyenne (équilibré : AVL, Rouge-Noir).               |
| <b>Union-Find (Disjoint Set)</b>        | Gère des ensembles disjoints ; utilisé dans Kruskal. Quasi $O(1)$ avec compression de chemin.            |
| <b>Table de hachage</b>                 | Accès moyen en $O(1)$ via une fonction de hachage ; gestion des collisions (chaînage, adressage ouvert). |
| <b>Trie (arbre préfixe)</b>             | Structure pour stocker des chaînes, recherche de préfixes en $O(\text{longueur du mot})$ .               |

### Union-Find : structure clé pour les graphes

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rang = [0] * n

    def trouver(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.trouver(self.parent[x]) # compression
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.trouver(x), self.trouver(y)
        if rx == ry:
            return
        if self.rang[rx] < self.rang[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rang[rx] == self.rang[ry]:
            self.rang[rx] += 1
```

## 7. Complexité algorithmique et NP-complétude

Certains problèmes n'admettent, à ce jour, aucun algorithme connu résolvant toutes leurs instances en temps polynomial. La théorie de la complexité les classe pour orienter le choix d'une stratégie (exacte, approchée, heuristique).

|                     |                                                                                    |
|---------------------|------------------------------------------------------------------------------------|
| <b>P</b>            | Problèmes solubles en temps polynomial (ex : tri, plus court chemin).              |
| <b>NP</b>           | Problèmes dont une solution proposée peut être vérifiée en temps polynomial.       |
| <b>NP-complet</b>   | Les problèmes les plus « durs » de NP (ex : voyageur de commerce, SAT, sac à dos). |
| <b>NP-difficile</b> | Au moins aussi difficile que NP-complet, sans forcément appartenir à NP.           |

### Stratégies face à un problème NP-difficile

- Algorithmes d'approximation : garantissent une solution proche de l'optimum en temps raisonnable.
- Heuristiques et métaheuristiques : recuit simulé, algorithmes génétiques, recherche locale.
- Programmation dynamique ou branch-and-bound : exacts mais limités aux petites instances.
- Réduction à un problème connu pour réutiliser un solveur existant (ex : solveur SAT).

### Pour aller plus loin

- Pratiquer sur des plateformes : Codeforces, LeetCode, Codingame, AtCoder.
- Étudier des ouvrages de référence : « Introduction to Algorithms » (Cormen et al.).
- Approfondir la théorie de la complexité (classes P vs NP, réductions polynomiales).
- Explorer les structures avancées : arbres de segments, Fenwick trees, arbres AVL/Rouge-Noir.

---

*Fin du guide — bonne pratique de l'algorithmique avancée !*